

UNIVERSIDAD TECNOLÓGICA NACIONAL

Facultad Regional Mendoza — Rectorado

Tecnicatura Universitaria en Programación a Distancia

TRABAJO PRÁCTICO N.º 1

Del dominio al esquema: modelo entidad-relación, modelo relacional, normalización y DDL en PostgreSQL

Proyecto Integrador: Food Store — Sistema de Gestión de Pedidos (Semana 1)

Materia:	Base de Datos I / Base de Datos 2
Comisión:	1PRO4
Integrantes:	• Lucas Avila • Amanda Pagano • Mateo Lautaro Liendo
Carrera:	Tecnicatura Universitaria en Programación
Institución:	Universidad Tecnológica Nacional (UTN FRM)
Fecha de Entrega:	Agosto de 2026
Entregables:	Diagrama ER (PNG/PDF/DBML), Documento Académico (PDF), schema.sql

1. Introducción y Dominio del Problema

El presente trabajo práctico conforma la primera entrega del proyecto integrador **Food Store**, un sistema transaccional de gestión de pedidos gastronómicos orientado a reemplazar registros planos e informales por una base de datos relacional robusta, normalizada y optimizada en **PostgreSQL**.

El desarrollo cubre la totalidad de las etapas de ingeniería de datos: modelado conceptual (MER), derivación formal al Modelo Relacional, demostración matemática de Normalización (1FN, 2FN, 3FN y BCNF), e implementación del script DDL (schema.sql) con tipos nativos, restricciones declarativas, integridad referencial y estrategias de indexación.

Reglas de Negocio fijadas por el dominio (R1 a R7):

- **R1:** Todo producto pertenece exactamente a una categoría; una categoría puede tener muchos productos, o ninguno todavía.
- **R2:** Todo pedido pertenece exactamente a un cliente registrado; un cliente puede no haber hecho ningún pedido aún, o haber hecho varios.
- **R3:** Un pedido puede incluir varios productos distintos, y un mismo producto puede aparecer en muchos pedidos distintos (relación N:M).
- **R4:** De cada producto dentro de un pedido interesa registrar cantidad y precio unitario histórico pactado al momento de facturar, inmune a aumentos futuros.
- **R5:** Un producto no puede tener stock ni precio negativos (restricciones declarativas de no negatividad).
- **R6:** Cada cliente tiene un correo electrónico único que lo identifica unívocamente en el sistema.
- **R7:** Política de baja lógica: el negocio no elimina físicamente productos ni categorías dados de baja; se conservan marcados como inactivos (activo = FALSE) para proteger la trazabilidad histórica de pedidos.

2. Parte 1: Modelo Entidad-Relación (MER)

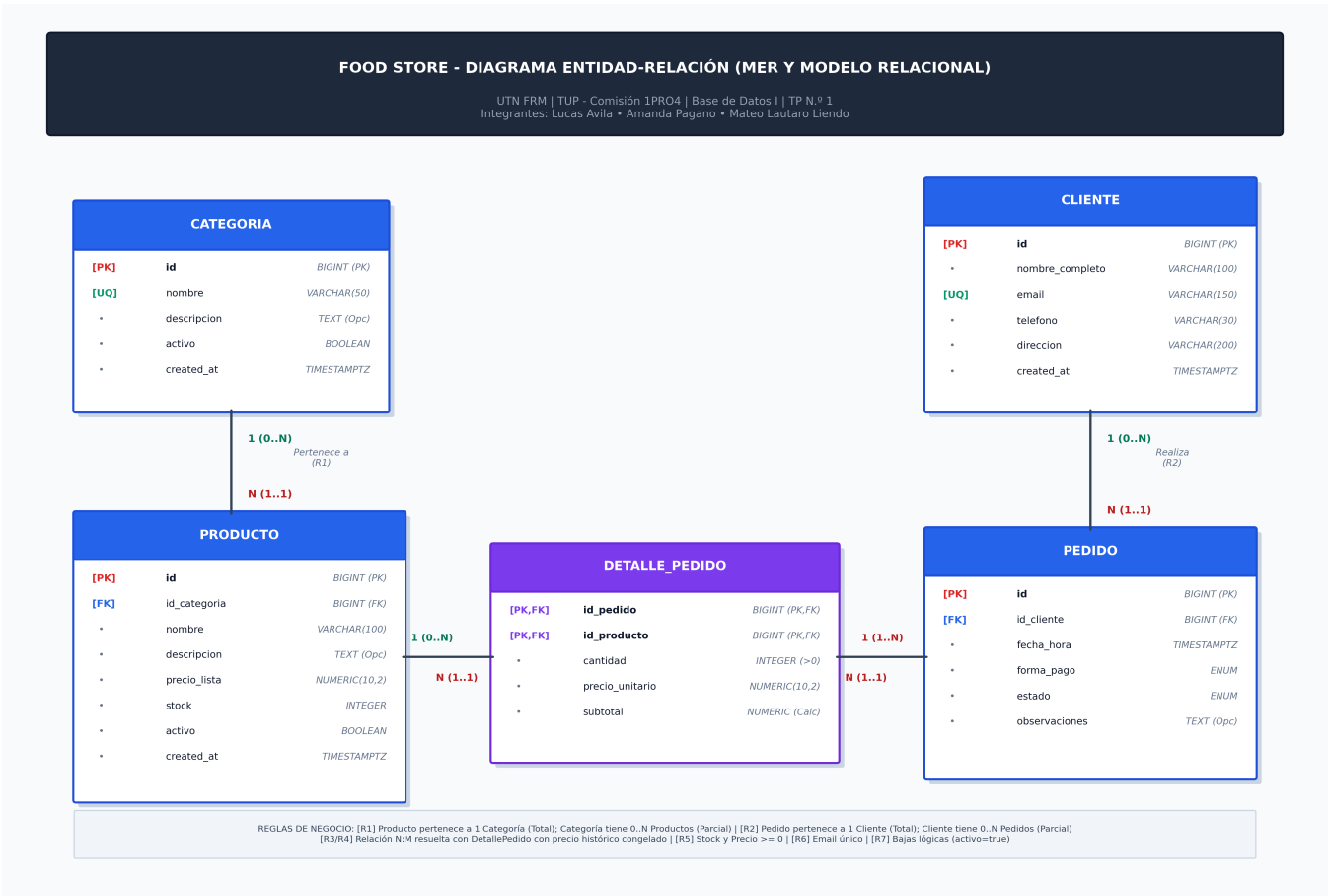
Se construyó el modelo conceptual formal respetando las reglas de negocio y estructurando las entidades fuertes y la entidad asociativa que resuelve la relación N:M.

2.1. Diccionario de Entidades y Atributos

Entidad	Atributo	Tipo Conceptual	Clave / Rol	Descripción / Regla de Negocio
Categoria	id	Integer / BigInt	PK	Identificador único de categoría
	nombre	Varchar(50)	Candidata / UQ	Nombre único del rubro (Pizzas, Bebidas)
	descripcion	Text	Atributo simple	Detalle opcional de la categoría
	activo	Boolean	Atributo simple	Indicador de baja lógica (R7, Default: TRUE)
	created_at	Timestamp	Auditoría	Marca temporal de creación
Cliente	id	Integer / BigInt	PK	Identificador único del cliente
	nombre_completo	Varchar(100)	Atributo simple	Nombre y apellido del cliente
	email	Varchar(150)	Candidata / UQ	Correo electrónico único (R6)
	telefono	Varchar(30)	Atributo simple	Contacto telefónico opcional
	direccion	Varchar(200)	Atributo simple	Dirección de entrega opcional
	created_at	Timestamp	Auditoría	Fecha y hora de registro
Producto	id	Integer / BigInt	PK	Identificador único del producto
	id_categoria	Integer / BigInt	FK	Referencia obligatoria a Categoría (R1)
	nombre	Varchar(100)	Atributo simple	Nombre comercial del producto
	precio_lista	Decimal(10,2)	Atributo simple	Precio actual de venta (≥ 0 , R5)
	stock	Integer	Atributo simple	Stock disponible (≥ 0 , R5)
	activo	Boolean	Atributo simple	Indicador de baja lógica (R7, Default: TRUE)
	created_at	Timestamp	Auditoría	Fecha y hora de registro
Pedido	id	Integer / BigInt	PK	Identificador único del pedido
	id_cliente	Integer / BigInt	FK	Referencia obligatoria a Cliente (R2)
	fecha_hora	Timestamp	Atributo simple	Momento de emisión de la orden
	forma_pago	Enumerado	Atributo simple	EFFECTIVO, TARJETA, TRANSFERENCIA
	estado	Enumerado	Atributo simple	PENDIENTE, EN_PREPARACION, etc.
DetallePedido	id_pedido	Integer / BigInt	PK / FK	Referencia al pedido
	id_producto	Integer / BigInt	PK / FK	Referencia al producto vendido
	cantidad	Integer	Atributo propio	Unidades compradas (> 0 , R4)
	precio_unitario	Decimal(10,2)	Atributo propio	Precio histórico congelado (R4)
	subtotal	Decimal(10,2)	Derivado / Calc	Calculado: cantidad * precio_unitario

2.2. Diagrama Entidad-Relación

A continuación se presenta el Diagrama Entidad-Relación completo en notación Crow's Foot y compatible con dbdiagram.io:



• **Pregunta 3: ¿Alguno de tus atributos podría descomponerse en partes más simples (por ejemplo, un nombre completo en nombre y apellido)? Justificá tu decisión de dejarlo compuesto o de separarlo.**

Respuesta: El atributo nombre_completo en Cliente es conceptualmente divisible en nombre y apellido. En sistemas de gran escala, descomponerlo permite búsquedas alfabéticas precisas, ordenamientos por apellido y personalización de mensajes. En este modelo inicial se mantiene como cadena simple por simplicidad operativa y concordancia con la planilla original. En cuanto al email, al ser un atributo atómico con restricción UNIQUE (R6), funciona como una excelente clave candidata alternativa.

3. Parte 2: Derivación al Modelo Relacional

Se aplicaron formalmente las reglas de pasaje de MER a Modelo Relacional:

- **Entidades Fuertes:** Se traducen en tablas con su respectiva clave primaria: Categoría, Cliente, Producto y Pedido.
- **Relaciones 1:N:** La clave primaria del lado '1' se propaga como clave foránea (FK) en la tabla del lado 'N'. Debido a la participación total de Producto y Pedido, las FKs `id_categoria` en `producto` e `id_cliente` en `pedido` son **NOT NULL**.
- **Relación N:M:** Se resuelve con la tabla intermedia (entidad asociativa) `DetallePedido`, conteniendo ambas FKs (NOT NULL) y los atributos de la relación: `cantidad` y `precio_unitario`.

3.1. Esquema Relacional Formal

categoria (id, nombre, descripcion, activo, created_at)

PK: id | **Clave Candidata / UNIQUE:** nombre

cliente (id, nombre_completo, email, telefono, direccion, created_at)

PK: id | **Clave Candidata / UNIQUE:** email (R6)

producto (id, nombre, descripcion, precio_lista, stock, activo, created_at, *id_categoria* → categoria)

PK: id | **FK:** *id_categoria* → categoria(id) [NOT NULL]

pedido (id, fecha_hora, forma_pago, estado, observaciones, *id_cliente* → cliente)

PK: id | **FK:** *id_cliente* → cliente(id) [NOT NULL]

detalle_pedido (*id_pedido* → pedido, *id_producto* → producto, cantidad, precio_unitario, subtotal)

PK Compuesta: (id_pedido, id_producto)

FK1: id_pedido → pedido(id) [NOT NULL, ON DELETE CASCADE]

FK2: id_producto → producto(id) [NOT NULL, ON DELETE RESTRICT]

3.2. Elección de Clave Primaria en la Tabla Intermedia

Se adoptó la **clave primaria compuesta (id_pedido, id_producto)**. Esta decisión modela con exactitud la regla de negocio que establece que un producto no se repite en más de una línea dentro del mismo pedido (si se añaden unidades, se modifica la cantidad).

3.3. Respuestas a las Preguntas Guía de la Parte 2

- **Pregunta 1:** ¿Qué pasaría con la integridad de tus datos si la tabla intermedia no incluyera ambas claves foráneas como NOT NULL?

Respuesta: Se destruiría la integridad referencial y la consistencia del modelo. Por la regla de **Integridad de Entidad**, ningún componente de una clave primaria compuesta puede aceptar valores nulos. Si `id_pedido` fuese NULL, tendríamos líneas de detalle 'huérfanas' sin orden asociada; si `id_producto` fuese NULL, se cobrarían montos y cantidades por productos inexistentes, arruinando el stock y la facturación.

- **Pregunta 2:** Si el pedido pudiera existir sin ningún producto asociado (por ejemplo, un pedido recién iniciado), ¿cambia esto la participación que definiste en la Parte 1 para la relación con la tabla intermedia? Explica el impacto.

Respuesta: Sí. La participación de **Pedido** en la relación con `DetallePedido` pasaría de ser **total (1..N)** a ser **parcial (0..N)**. En el motor relacional esto es totalmente viable, ya que la tabla `pedido` no tiene FK hacia `detalle_pedido`. El impacto radica en que la aplicación puede crear un pedido en borrador o 'carrito abierto' antes de insertar sus productos. No obstante, para facturar o enviar a cocina, una validación a nivel de aplicación o trigger debe exigir al menos un producto.

4. Parte 3: Normalización hasta 3FN y BCNF

A continuación se presenta el desarrollo formal paso a paso del proceso de normalización sobre la planilla de ventas original:

N.º Pedido	Fecha	Cliente	Producto	Categoría	Precio Unit.	Cant.	Subtotal	Forma Pago
1	01/03/2026	Ana Gómez	Muzzarella	Pizzas	1000.00	2	2000.00	EFFECTIVO
1	01/03/2026	Ana Gómez	Coca 1.5L	Bebidas	800.00	1	800.00	EFFECTIVO
2	01/03/2026	Luis Paz	Napolitana	Pizzas	1500.00	1	1500.00	TARJETA
3	05/03/2026	Ana Gómez	Muzzarella	Pizzas	1050.00	3	3150.00	TRANSFERENCIA
4	06/03/2026	Marta Ruiz	Coca 1.5L	Bebidas	800.00	4	3200.00	EFFECTIVO
4	06/03/2026	Marta Ruiz	Napolitana	Pizzas	1500.00	2	3000.00	EFFECTIVO
5	07/03/2026	Luis Paz	Muzzarella	Pizzas	1050.00	1	1050.00	TARJETA

Paso 1: Identificación de la Clave Candidata Universal

En la relación universal $R(\text{Nro_pedido}, \text{Fecha}, \text{Cliente}, \text{Producto}, \text{Categoría}, \text{Precio_unitario}, \text{Cant}, \text{Subtotal}, \text{Forma_pago})$:

- **Nro_pedido** no es clave por sí solo (se repite en pedidos con varios ítems, ej. pedidos 1 y 4).
- **Producto** no es clave por sí solo (se repite en distintos pedidos a lo largo del tiempo).
- Dado que dentro de un mismo pedido un producto no se repite, el conjunto **{Nro_pedido, Producto}** identifica de manera única a cada fila.

Clave Candidata: {Nro_pedido, Producto}

Paso 2: Deducción de Dependencias Funcionales (DF)

A partir de las reglas del dominio y la observación empírica de los datos:

- **DF1:** $\text{Nro_pedido} \rightarrow \text{Fecha}, \text{Cliente}, \text{Forma_pago}$ (Un pedido tiene una única fecha, cliente y forma de pago).
- **DF2:** $\text{Producto} \rightarrow \text{Categoría}$ (Cada producto pertenece exactamente a una categoría).
- **DF3:** $\{\text{Nro_pedido}, \text{Producto}\} \rightarrow \text{Cant}, \text{Precio_unitario}, \text{Subtotal}$ (En un pedido específico, para un producto dado, se pacta una cantidad y un precio de venta).
- **DF4:** $\{\text{Cant}, \text{Precio_unitario}\} \rightarrow \text{Subtotal}$ (Dependencia funcional matemática: $\text{subtotal} = \text{cant} * \text{precio_unitario}$).

Observación sobre Precio_unitario: En los datos se observa que la Muzzarella cuesta \$1000 en el pedido 1 (01/03) y \$1050 en el pedido 3 (05/03). Por ende, $\text{Producto} \nrightarrow \text{Precio_unitario}$; el precio facturado depende de la compra puntual $\{\text{Nro_pedido}, \text{Producto}\}$.

Paso 3: Verificación de Primera Forma Normal (1FN)

Condición 1FN: Atomicidad absoluta de valores (sin grupos repetitivos ni arrays en celdas).

Evaluación: Cada celda contiene un único valor escalar atómico.

Conclusión: La relación universal **CUMPLE 1FN**.

Paso 4: Verificación y Descomposición a Segunda Forma Normal (2FN)

Condición 2FN: Está en 1FN y ningún atributo no clave depende parcialmente de una parte de la clave compuesta $\{\text{Nro_pedido}, \text{Producto}\}$.

Detección de Violaciones (Dependencias Parciales):

- Fecha, Cliente, Forma_pago dependen solo de Nro_pedido (subconjunto propio de la clave).
- Categoría depende solo de Producto (subconjunto propio de la clave).

Descomposición en 2FN:

- **R_PEDIDO** (Nro_pedido, Fecha, Cliente, Forma_pago)
- **R_PRODUCTO** (Producto, Categoría)
- **R_DETALLE_PEDIDO** (Nro_pedido, Producto, Cant, Precio_unitario, Subtotal)

Paso 5: Verificación y Descomposición a Tercera Forma Normal (3FN)

Condición 3FN: Está en 2FN y no existen dependencias transitivas entre atributos no clave.

Análisis: En R_DETALLE_PEDIDO, el atributo Subtotal depende funcionalmente de {Cant, Precio_unitario} (DF4). Como ninguno de ellos es clave candidata, Subtotal es un atributo derivado redundante que introduce transitividad.

Resolución 3FN: Se elimina Subtotal del almacenamiento físico persistente.

- **R_PEDIDO** (Nro_pedido, Fecha, Cliente, Forma_pago)
- **R_PRODUCTO** (Producto, Categoría)
- **R_DETALLE_PEDIDO** (Nro_pedido, Producto, Cant, Precio_unitario)

Paso 6: Verificación de Forma Normal de Boyce-Codd (BCNF)

Condición BCNF: Para toda dependencia funcional no trivial $X \twoheadrightarrow Y$, el determinante X debe ser superclave.

- En R_PEDIDO: El determinante Nro_pedido es clave primaria.
- En R_PRODUCTO: El determinante Producto es clave primaria.
- En R_DETALLE_PEDIDO: El determinante {Nro_pedido, Producto} es clave primaria compuesta.

Conclusión: Todo determinante es superclave. El esquema **CUMPLE BCNF** sin requerir divisiones adicionales.

Paso 7: Conjunto Final de Tablas Normalizadas

1. **PEDIDO** (nro_pedido, fecha, cliente, forma_pago)
2. **PRODUCTO** (producto, categoria)
3. **DETALLE_PEDIDO** (nro_pedido, producto, cant, precio_unitario)
 - FK1: nro_pedido → PEDIDO(nro_pedido)
 - FK2: producto → PRODUCTO(producto)

4.1. Respuestas a las Preguntas de Integración de la Parte 3

- **Pregunta 1: ¿Qué nos dice la diferencia entre modelar desde el enunciado vs normalizar datos históricos?**

Respuesta: La normalización sobre datos históricos (Bottom-Up) es un método analítico que limpia redundancias y anomalías sobre datos que ya se guardaron en el pasado, pero no puede inferir requerimientos que no estén presentes en la muestra (como emails únicos, control de stock o bajas lógicas). En cambio, el modelado conceptual desde el enunciado (Top-Down) permite anticipar la arquitectura completa del negocio, restricciones de integridad y necesidades futuras. Ambos enfoques se complementan para validar el diseño.

- **Pregunta 2: ¿Conviene almacenar subtotal? Justificá.**

Respuesta: Almacenar subtotal como columna modificable viola la 3FN por redundancia de atributo derivado y riesgo de inconsistencias. La mejor práctica es calcularlo al vuelo en consultas/vistas o definirlo en PostgreSQL como columna generada (GENERATED ALWAYS AS (cantidad * precio_unitario) STORED), asegurando consistencia matemática sin anomalías.

5. Parte 4: Implementación DDL en PostgreSQL (schema.sql)

El script schema.sql fue codificado y verificado para PostgreSQL 14+, implementando todas las restricciones y optimizaciones:

5.1. Decisiones de Diseño e Índices Fundamentados

Nombre del Índice	Tabla y Columnas	Tipo / Filtro	Consulta Acelerada / Justificación
idx_pedidos_cliente_id	pedido(id_cliente)	B-Tree estándar	Acelera la consulta frecuente de historial de pedidos por cliente ('Mis Pedidos').
idx_productos_categoria_activo	producto(id_categoria, activo)	Parcial (WHERE activo = TRUE)	Acelera la renderización del menú de venta filtrando ítems disponibles por rubro.
idx_detalle_pedido_producto_id	detalle_pedido(id_producto)	B-Tree estándar	Acelera reportes agregados de recaudación y ventas históricas por producto.

5.2. Estructura Principal del Script DDL (schema.sql)

```
-- ENUMS CREATE TYPE forma_pago_enum AS ENUM ('EFECTIVO', 'TARJETA', 'TRANSFERENCIA'); CREATE TYPE estado_pedido_enum AS ENUM ('PENDIENTE', 'EN_PREPARACION', 'ENTREGADO', 'CANCELADO'); -- TABLAS PRINCIPALES CREATE TABLE categoria ( id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY, nombre VARCHAR(50) NOT NULL UNIQUE, descripcion TEXT NULL, activo BOOLEAN NOT NULL DEFAULT TRUE, created_at TIMESTAMPTZ NOT NULL DEFAULT now(), CONSTRAINT chk_categoria_nombre_no_vacio CHECK (trim(nombre) <> '' ) ); CREATE TABLE cliente ( id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY, nombre_completo VARCHAR(100) NOT NULL, email VARCHAR(150) NOT NULL UNIQUE, telefono VARCHAR(30) NULL, direccion VARCHAR(200) NULL, created_at TIMESTAMPTZ NOT NULL DEFAULT now(), CONSTRAINT chk_cliente_email_valido CHECK (email ~* '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$' ) ); CREATE TABLE producto ( id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY, id_categoria BIGINT NOT NULL REFERENCES categoria(id) ON DELETE RESTRICT, nombre VARCHAR(100) NOT NULL, descripcion TEXT NULL, precio_lista NUMERIC(10, 2) NOT NULL CHECK (precio_lista >= 0.00), stock INTEGER NOT NULL DEFAULT 0 CHECK (stock >= 0), activo BOOLEAN NOT NULL DEFAULT TRUE, created_at TIMESTAMPTZ NOT NULL DEFAULT now() ); CREATE TABLE pedido ( id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY, id_cliente BIGINT NOT NULL REFERENCES cliente(id) ON DELETE RESTRICT, fecha_hora TIMESTAMPTZ NOT NULL DEFAULT now(), forma_pago forma_pago_enum NOT NULL, estado estado_pedido_enum NOT NULL DEFAULT 'PENDIENTE', observaciones TEXT NULL ); CREATE TABLE detalle_pedido ( id_pedido BIGINT NOT NULL REFERENCES pedido(id) ON DELETE CASCADE, id_producto BIGINT NOT NULL REFERENCES producto(id) ON DELETE RESTRICT, cantidad INTEGER NOT NULL CHECK (cantidad > 0), precio_unitario NUMERIC(10, 2) NOT NULL CHECK (precio_unitario >= 0.00), subtotal NUMERIC(10, 2) GENERATED ALWAYS AS (cantidad * precio_unitario) STORED, CONSTRAINT pk_detalle_pedido PRIMARY KEY (id_pedido, id_producto) );
```

6. Declaración de Uso de Inteligencia Artificial (DUIA)

En cumplimiento de los lineamientos pedagógicos de la UTN FRM (Comisión 1PRO4), se formaliza la DUIA grupal:

Dimensión	Detalle y Justificación Grupal
Herramienta Utilizada	Gemini Spark (asistente de investigación y modelado de datos).
Alcance de la Asistencia	Verificación formal de dependencias funcionales, estructuración del diccionario de datos, renderización vectorial del diagrama ER y generación del script DDL.
Protocolo de Seguridad	Copia → Transacción → Respaldo: Validación idempotente y transaccional del script SQL.
Supervisión Humana	Revisión colaborativa del grupo (Avila, Pagano, Liendo) sobre cada restricción de integridad y consistencia con las reglas R1 a R7.

7. Conclusiones y Bibliografía

El diseño obtenido para **Food Store** cumple cabalmente con las especificaciones académicas y técnicas de la cátedra para la Semana 1. El esquema relacional resultante se encuentra en **Forma Normal de Boyce-Codd (BCNF)**, libre de redundancias y anomalías de actualización.

- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database System Concepts* (7th ed.). McGraw-Hill.
- Elmasri, R., & Navathe, S. B. (2015). *Fundamentals of Database Systems* (7th ed.). Pearson.
- PostgreSQL Global Development Group. (2026). *PostgreSQL 16 Documentation: DDL & Constraints*.